UTM
UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING
UTM Johor Bahru

SECJ 3032: Software Engineering FYP1

Semester 01, 2024/2025

Software Requirements Specification (SRS)

<<Project Title>>

<<Version No.>>

Prepared by: << Name >>

Revision Page

a. Overview

Describe the content of the current version of SRS (see below – the note in red).

b. Target Audience

State the targeted audience for the SRS.

c. Version Control History

Version	Primary Author(s)	Description of Version	Date Completed
1.0	Team leader 1 (Full Name)	Completed Chapter X, Section…	dd/mm/yyyy
…			

Note:
This Software Requirements Specification (SRS) template is adapted from IEEE Recommended Practice for Software Requirements Specification (SRS) (IEEE Std. 830-1998) that is simplified and customized to meet the need of SECJ3203 FYP1 SE course at Faculty of Computing, UTM. Examples of models are from Arlow and Neustadt (2002) and other sources stated accordingly. This template is tailored for SRS in a plan-driven software development approach.

Table of Contents

Update this table of content accordingly

1. Introduction

The following sections of the Software Requirements Specification (SRS) document should provide the details of the entire document. Remove the notes in red texts when submitting including these notes.

1.1 Purpose

This subsection should:
a) *Delineate the purpose of the SRS;*
b) *Specify the intended audience for the SRS.*

This SRS describes…

1.2 Scope

This subsection should:
a) Identify the software product(s) to be produced by name (e.g., Host DBMS, Report Generator, etc.);
b) Explain what the software product(s) will, and, if necessary, will not do;
c) *Describe the application of the software being specified, including relevant benefits, objectives, and goals;*
d) Be consistent with similar statements in higher-level specifications (e.g., the system requirements specification), if they exist.

The software product is…

1.3 Definitions, Acronyms and Abbreviation

This subsection should provide the definitions of all terms, acronyms, and abbreviations used in the SRS.

Definitions of all terms, acronyms and abbreviations used are to be defined here.

1.4 References

This subsection should:
a) *Provide a complete list of all documents referenced elsewhere in the SRS;*
b) Identify each document by title, report number (if applicable), date, and publishing organization;
c) Specify the sources from which the references can be obtained.

Specify a complete list of references using a standardized reference format.

1.5 Overview

This subsection should:
a) *Describe what the rest of the SRS contains;*
b) *Explain how the SRS is organized.*

Give an overview of the content of this SRS document.

2. Specific Requirements

This section 2 of SRS should contain all the software requirements to a level of detail sufficient to enable designers to design a system to satisfy those requirements, and testers to test that the system satisfies those requirements. Throughout this section, every stated requirement should be externally perceivable by users, operators, or other external systems. These requirements should ***include at a minimum a description of every input (stimulus) into the system, every output (response) from the system, and all functions performed by the system*** *in response to an input or in support of an output.*

2.1 User characteristics

This should specify the following:
In an SRS, the user characteristics section describes the intended users of the system and their characteristics, such as their technical expertise, knowledge, and physical abilities. This section is important because it provides the development team with a clear understanding of whom the system is designed for, and helps to ensure that the system is developed to meet the needs of its intended users.
Example. Describe how the system will interact with its users.
The "Emotion Chatbot Analyzer" software will be used by two main types of users: UTM students and UTM counselors.
2.1.1 UTM Students

- *The UTM students who will use the software are expected to have basic computer skills, including familiarity with web-based applications.*
- *They may have varying levels of emotional intelligence and may experience a wide range of emotional states, from mild stress to severe anxiety or depression.*
- *Some students may be reluctant to seek counseling or may not have access to professional ccounseling services, making the chatbot a valuable resource for emotional support and guidance.*

2.1.1 UTM Counsellors

- *The UTM counsellors who will use the software are expected to have professional training and experience in counselling and psychology.*
- *They may have varying levels of technical expertise, but will receive training on how to use the software.*
- *They will use the software to access student emotion records and provide counselling services to students as needed.*
- *Both types of users will be required to provide personal information such as name and email address in order to register for the software. The software will be designed with privacy and security measures in place to protect user information.*

2.2 System Features

Specify the product perspective.

Example.
The Emotion Chatbot Analyzer is a software system that operates on mobile devices, including both Android and iOS operating systems. The system provides a means for UTM students to better manage their emotional health and at the same time simplifies the process of seeking emotional support from UTM counselors The system also implemented a real-time cloud-based database for better user experiences.

Specify the product module and function..
Include use case diagram here – see an example of Emotion Chatbot Analyzer below
Then, provide the description of each module and function using a table.

Example.
The system features is illustrated in Figure X.X below. The detail description of each module and functions is tabulated in Table X.X.

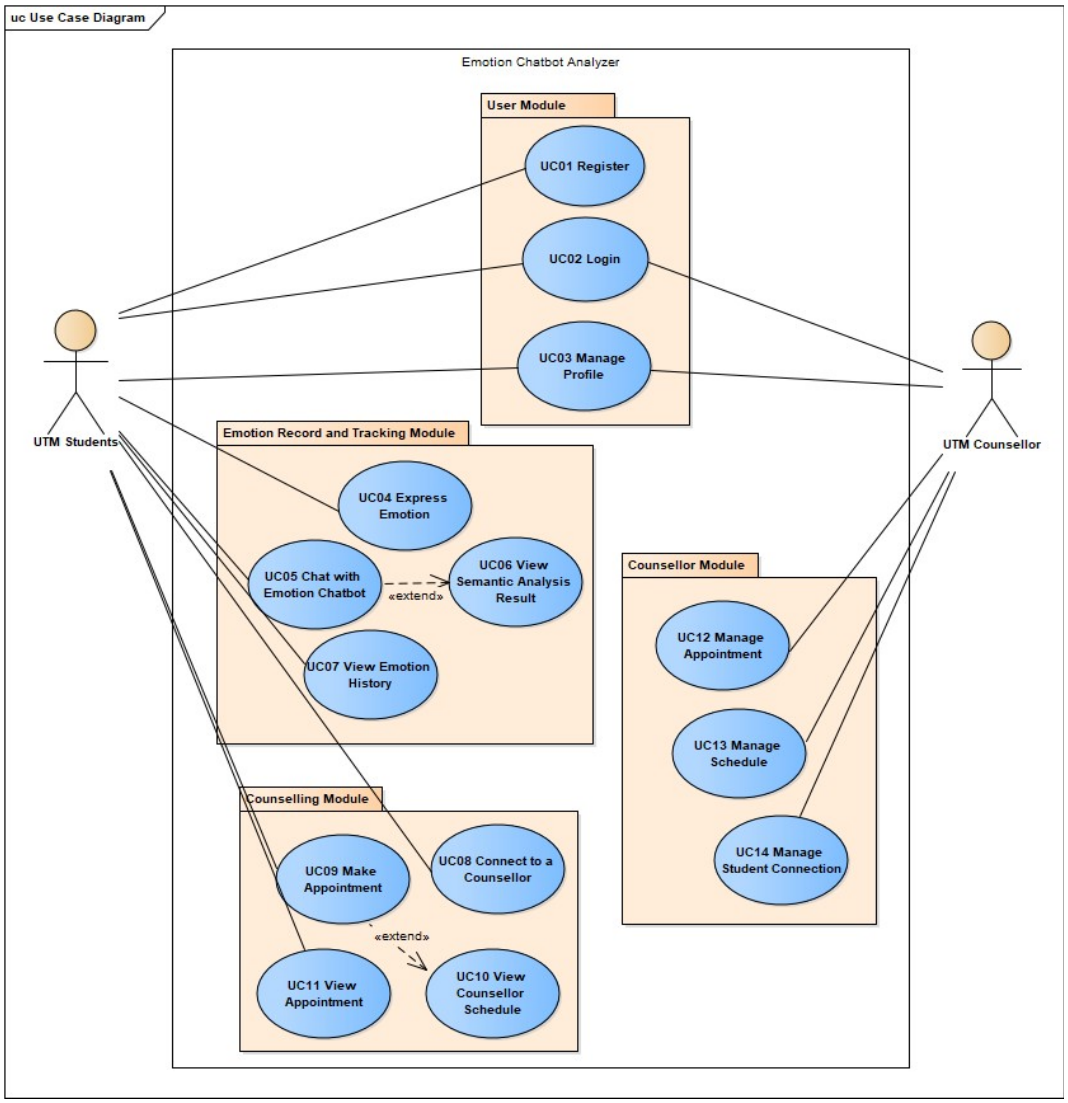

Figure X.X: Use Case Diagram for <Name of the System>

Table X.X: Description of Module and Functions for <Name of the System>

Module	Function	Description
User Module	*UC01 – Register*	*This use case allows students to sign up as a user for the mobile application.*

*[Include domain model i.e. **class diagram** without the operation part – only attributes without details on visibility and type, explain each class and its attributes including the relationships among the classes, see the example.]*

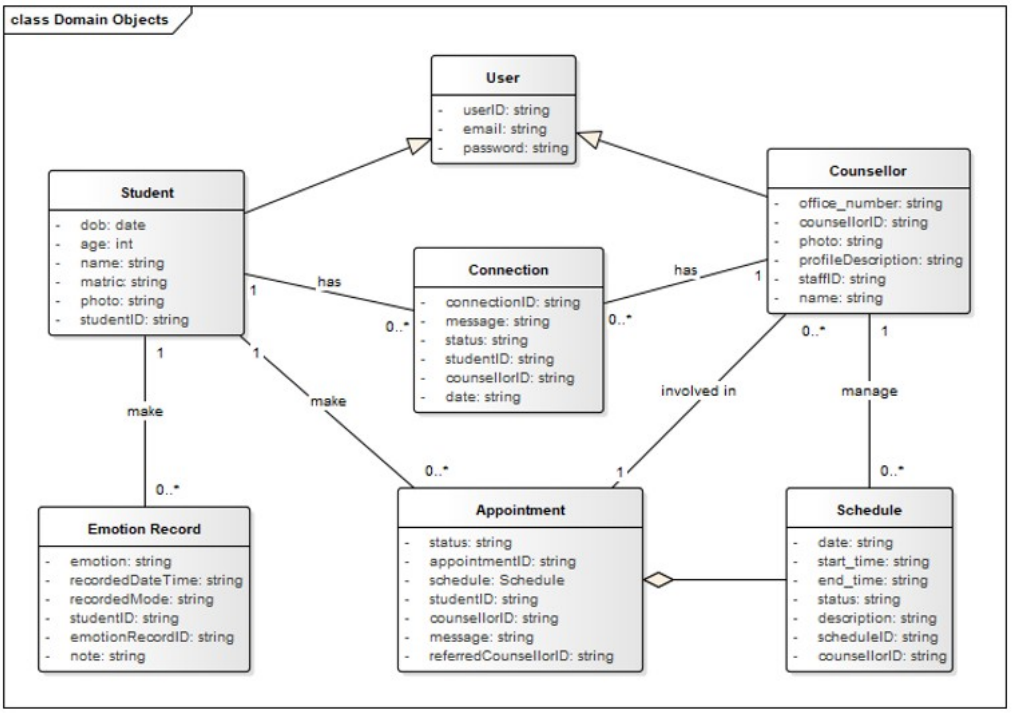

Figure X.X: Domain Model for <Name of the System>

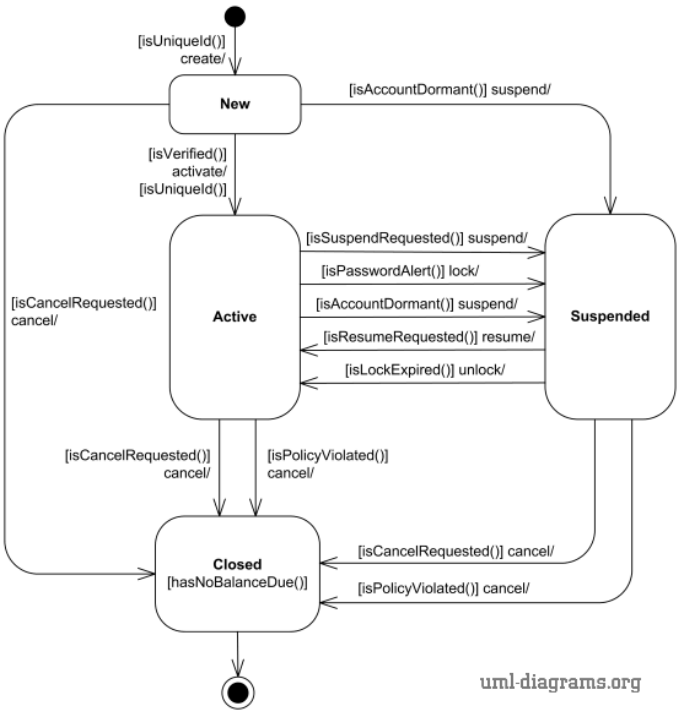

Figure X.X: State Diagram for <Name of the Selected Class>

2.3 Use Case Details

2.3.1 UC001: Use Case <Name of Use Case 1>

[Provide code for each use case such as UC001 and so on… Include Use Case Description for each use case as in the example below. If there are any alternative flows, add the rows accordingly. Otherwise, just remove the rows. Ensure each use case has its own unique ID and the name of the heading above corresponds to the name of use case in the use case diagram (refer to Figure X.X). Different scenarios of a use case require separate use case descriptions.]

Table X.X: Use Case Description for <Name of Use Case>

Use Case ID	*//Unique ID with UC<#>*
Use Case Name	*//Unique Name with <Action> <Verb> naming rule*
Description	*//Brief description about the this UC*
Actor(s)	*//List the primary i.e. direct/end-users, secondary i.e. external/interfacing systems (e.g. API) that shall trigger/involved in this use case – can use stickman symbol or alternative rectangle with <system> stereotype, can apply generalization for general vs specialized actors*
Pre-condition(s)	*//List/describe the constraint(s) that should be checked/met to initiate/execute this use case*
Normal Flow(s)- NF	*// Synonym: Main/Primary Scenario* *# The <actor> <perform action>* *# The <system> <response to actor action>*

	e.g.: 1. The user enter input no of item in the ordering form *2. The system validates the inputs….* *3. …*
Alternative Flow(s) - AF	*// Synonym: Alternative Scenario ~ when checking condition (s)/ decision (s) occur i.e. true/false, Boolean* *e.g.:* ***AF1. Cancel Operation***
Exception Flow(s) - EF	*// Synonym: Error handling Scenario ~ seldom relates to system mechanism to manage flow control of expected errors* *e.g.:* ***EF1. System Failure to Update Database***
Post-condition(s)	*//List the system states after use case ends/executed*

[Include sequence diagram and activity diagram for each respective user case. See example below. May consider including different scenarios in different diagrams, if necessary, to avoid clutter.]

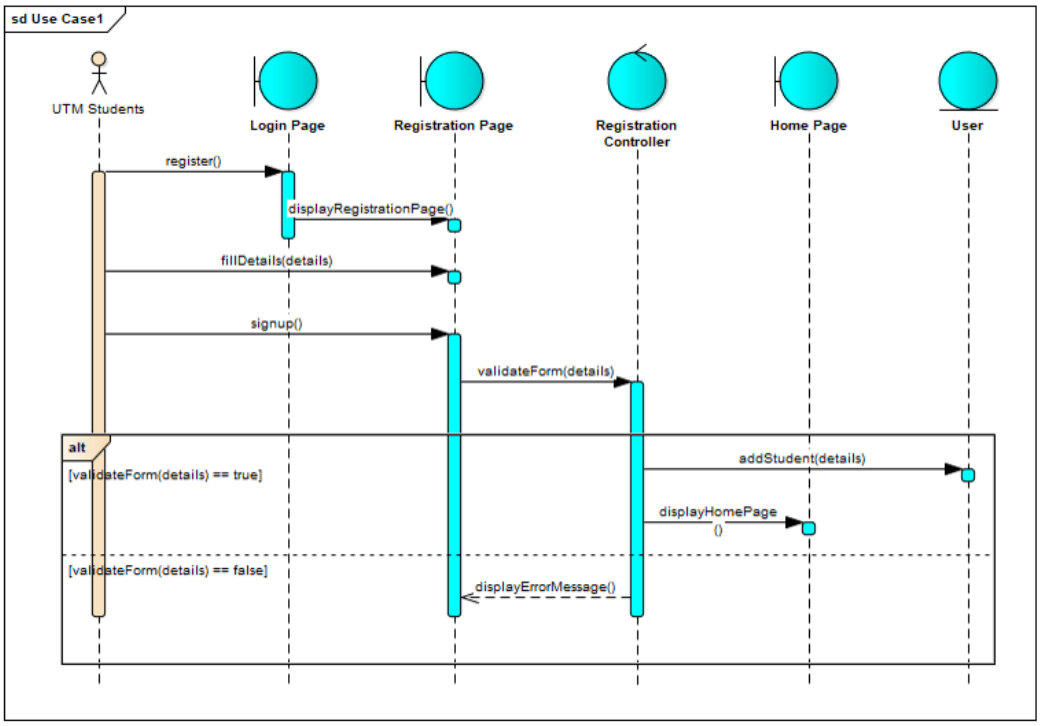

Figure X.X: Sequence Diagram for <Name of Use Case/Scenario if more than one scenario>

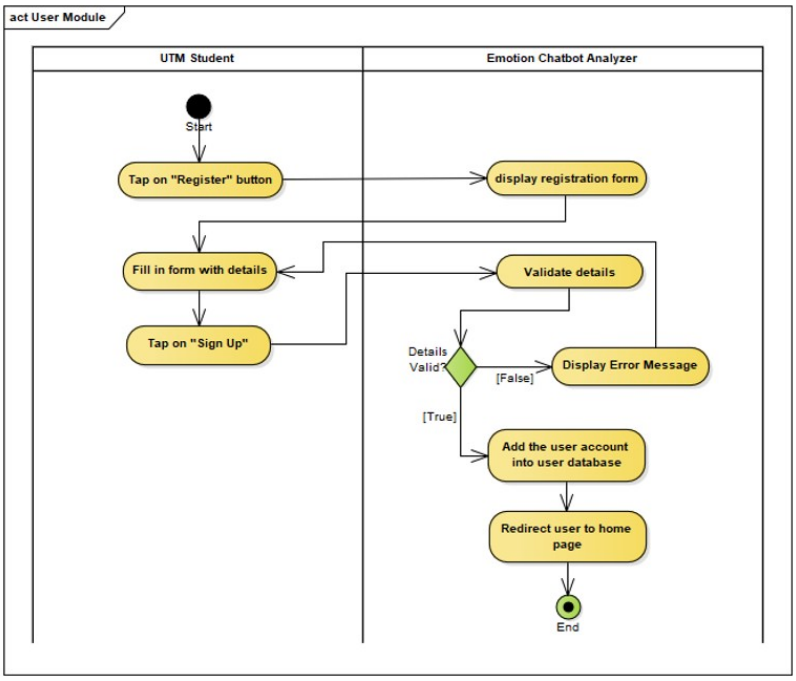

Figure X.X: Activity Diagram for <Name of Use Case/Scenario if more than one scenario>

2.3.2 UC002: Use Case <Name of Use Case 2>

2.3.3 UC*n*: Use Case <Name of Use Case *n*>

…

2.4 Performance and Other Requirements

Non-functional requirements are the requirements that describe how the system should behave or operate, rather than what it should do. They address the characteristics or qualities of the system, such as its usability, reliability, performance, security, maintainability, and compatibility. Non-functional requirements are important to write in the SD because they help ensure that the system meets the stakeholders' needs and expectations.The ISO/IEC/IEEE 29148 standard suggests that non-functional requirements should be specified under three main categories: Software System Attributes, Performance and Other Requirements.

Example of Software System Atrributes are as follows:
Software system attributes define the overall qualities or characteristics of the software. These attributes are the foundation on which the software is built, and they include the following:

- *Usability: The ease with which users can interact with the system.*
- *Reliability: The ability of the system to perform its functions correctly and consistently.*
- *Maintainability: The ease with which the system can be modified, repaired, and enhanced.*
- *Portability: The ability of the system to run on different hardware and software platforms.*
- *Compatibility: The ability of the system to work with other systems and components.*

Example of Performance is as follows:
Performance requirements define the system's capability to respond to user requests and handle data in a timely manner. These requirements include the following:

- *Response Time: The time it takes for the system to respond to a user request.*
- *Throughput: The number of requests the system can handle in a given period of time.*
- *Capacity: The maximum number of users or amount of data that the system can handle.*
- *Availability: The percentage of time the system is operational and accessible to users.*

Example of Other Requirements is as follows:
Other requirements refer to any non-functional requirements that do not fit under the categories of software system attributes or performance. These requirements include the following:

- *Security: The protection of the system and its data from unauthorized access and malicious attacks.*
- *Safety: The ability of the system to operate safely without causing harm to users or the environment.*
- *Legal and Regulatory: Compliance with laws, regulations, and standards.*
- *Environmental: The impact of the system on the environment.*

2.5 Design Constraints

Explain any constraints imposed by the organization where the software product will be used such as the system must adhere to certain organizational standard and other external non-functional requirements.

Example is as follows:
Environmental constraints: These constraints relate to the physical environment in which the system will operate, such as temperature, humidity, and lighting conditions. An example of an environmental constraint is: "The system must be designed to operate in temperatures ranging from 0°C to 40°C."

Hardware constraints: These constraints relate to the specific hardware components that will be used in the system, such as processors, memory, and storage. An example of a hardware constraint is: "The system must be designed to run on a minimum of 8GB of RAM."

Security constraints: These constraints relate to the security requirements for the system, such as data encryption, authentication, and access control. An example of a security constraint is: "The system must be designed to encrypt all sensitive user data using AES-256 encryption."

Compatibility constraints: These constraints relate to the compatibility requirements for the system, such as compatibility with specific software or hardware components. An example of a compatibility constraint is: "The system must be designed to be compatible with Windows 10 operating system."

Performance constraints: These constraints relate to the performance requirements for the system, such as response time, throughput, and capacity. An example of a performance constraint is: "The system must be designed to support a maximum of 1000 concurrent users with a response time of less than 5 seconds."

Appendix A: <Evidence of Requirements Elicitation Artefacts>